

PDF Deep Dive — Python .ref

Purpose

This document is a **deep-dive** `.ref` knowledge capsule for learning how to work with PDFs in Python. It is intentionally hands-on: explanations, minimal examples you should retype, experiments to run, and notes on how the libraries are designed and why they behave the way they do. The goal is not just to get working code, but to make you *familiar with the library's API style, control flow, and common pitfalls* so you can build, debug, and extend confidently.

Quick map (what you'll learn)

1. Core concepts: PDF as a format, pages, streams, fonts, objects.
 2. Creating PDFs: `fpdf2` (simple) and `reportlab` (powerful). Learn the "minimal loop" for generation.
 3. Reading & manipulating: `PyPDF2` (renamed `pypdf` in newer versions) and `pdfplumber`.
 4. Image-based PDFs & OCR: `pdf2image` + `pytesseract`.
 5. Export pipeline: turning `.ref` -> `.md` -> `.pdf` (practical).
 6. Exercises: 1-feature-at-a-time with purposeful experiments.
 7. How to read library docs & source to learn effectively.
-

1. Core concepts: what is a

PDF? (short, but essential)

- PDF is a container format (not plain text). Internally it has objects: dictionaries, streams, arrays, numbers, names.
- A typical PDF has: header, body (objects), cross-reference table, trailer.
- Pages are objects that reference content streams (drawing instructions), fonts, resources, and metadata.
- Libraries wrap this complexity into friendly APIs: e.g., "add_page()" creates a new page object, "drawString" writes text to a coordinate, "output()" serializes to disk.

Keep this mental model: **high-level API calls -> library builds objects -> library serializes PDF structure.**

2. Creating PDFs (FPDF2)

FPDF2 is simple, imperative, and great to learn the "PDF generation loop". Install:

```
pip install fpdf2
```

Minimal working example (type it yourself)

```
from fpdf import FPDF

pdf = FPDF()          # create document object (in-memory model)
pdf.add_page()        # create page context (must do before writing)
pdf.set_font("Helvetica", size=12) # select font for future text
pdf.cell(0, 10, "Hello world!", ln=True) # write a single-line cell (auto width)
pdf.multi_cell(0, 8, "Multi-line text wraps automatically based on width.")
pdf.output("out_fpdf.pdf") # serialize document to file
```

Key functions & their semantics

- `FPDF()` — constructs a document object. Internally stores a list of pages and current state (font, color, margins).
- `add_page()` — starts a new page. Internally creates a Page object and sets a drawing cursor.
- `set_font(family, style='', size=...)` — sets font in internal state. FPDF uses font metrics to compute text widths and line heights.
- `cell(w, h, txt, ln=False)` — draws a rectangular cell; if `w=0` it uses the remaining width (auto-fit). `ln=True` moves cursor to next line.
- `multi_cell(w, h, txt)` — wraps text across lines, useful for paragraphs.
- `image(path, x=None, y=None, w=0, h=0)` — places an image; calculates aspect ratio if only one dimension provided.
- `output(filename)` — writes PDF bytes to disk; this is the serialization step.

Experiments (do these)

1. Remove `add_page()` and run — observe the error or silent failure. Understand why a page context is required.
2. Use very long text in `cell` vs `multi_cell` to see wrapping behavior.
3. Call `pdf.set_font("Times", size=20)` then `cell` — inspect text size difference.
4. Inspect `dir(pdf)` and `help(pdf.cell)` in a REPL to see implementations and docstrings.

Internal design notes (how fpdf thinks)

- FPDF keeps a **current cursor** (x, y). Most drawing functions alter or consult this cursor.
 - It also maintains a stack of resources (fonts, images) to embed in PDF resource objects at serialization.
 - When you call `output`, FPDF iterates pages, collects resources, writes objects, builds xref table, writes trailer.
-

3. Creating PDFs (ReportLab)

ReportLab is coordinate-based and more powerful. Install:

```
pip install reportlab
```

Minimal example

```
from reportlab.lib.pagesizes import A4
from reportlab.pdfgen import canvas

c = canvas.Canvas("out_reportlab.pdf", pagesize=A4)
width, height = A4
c.drawString(100, height - 100, "Hello from ReportLab!") # (x, y) from bottom-left
c.drawImage("logo.png", 50, height - 200, width=100, height=100)
c.showPage() # finish page
c.save() # write file
```

Key differences vs FPDF2

- ReportLab uses an explicit **coordinate system** (origin at bottom-left). You control exact positions.
- It exposes drawing primitives (lines, shapes, paths), text objects, and canvas state (save/restore).
- Better when you need charts, precise layout, or vector drawings.

Experiments

1. Draw the same text at y, y-10, y-20 — observe coordinate system.
 2. Create a small grid of rectangles using `rect` to understand units (points = 1/72 inch).
 3. Use `c.beginText()` and `text_obj.textLine()` to handle complex text flows.
-

4. Reading and extracting text (PyPDF2 / pypdf)

Note: The popular PyPDF2 project was forked/renamed to `pypdf`. Both names may appear. Install:

```
pip install pypdf pdfplumber
```

Basic PyPDF2 example (text & metadata)

```
from pypdf import PdfReader

reader = PdfReader("sample.pdf")
print("pages:", len(reader.pages))
page0 = reader.pages[0]
text = page0.extract_text() # NOT perfect: depends on PDF internal text layout
print(text)
print(reader.metadata)
```

What `extract_text()` does conceptually

- It parses content streams (which contain text-drawing operators).
- It collects characters and reconstructs text order heuristically using coordinates and text showing operators.
- PDFs don't store "plain paragraphs"; text might be placed letter-by-letter at arbitrary positions — so `extract_text()` must infer logical order, which is why results can be messy.

Manipulation (merge, split)

```
from pypdf import PdfWriter, PdfReader

r = PdfReader("a.pdf")
w = PdfWriter()
w.add_page(r.pages[0])
with open("single.pdf", "wb") as f: w.write(f)
```

Experiments

1. Inspect `reader.pages[0].extract_text()` on a multi-column PDF — see the column order problem.
 2. Try merging two PDFs, then open the result to verify page order.
 3. `help(reader)` and trace where `extract_text` calls parsing utilities (open source) to read a bit of its implementation.
-

5. High-quality extraction (pdfplumber)

`pdfplumber` wraps `pdfminer.six` and gives high-level abstractions for tables and positions.

```
import pdfplumber
with pdfplumber.open("sample.pdf") as pdf:
    for page in pdf.pages:
        print(page.extract_text())
        tables = page.extract_tables() # list of table rows
        for t in tables:
            print("table:", t)
```

Why use pdfplumber?

- It gives bounding boxes and coordinates for each text object, so you can reconstruct columns and tables more reliably.
- Useful for data extraction workflows (invoices, reports).

Experiments

1. Use `page.extract_words()` to see words with bbox coordinates.
 2. Create a rule that sorts words by y then x to attempt making better paragraphs.
-

6. Image-based PDFs and OCR

Scanned PDFs have no selectable text. Use `pdf2image` + `pytesseract` for OCR.

Install prerequisites:

```
pip install pdf2image pytesseract
# on Linux you also need poppler installed system-wide (e.g., apt install poppler-utils)
# and tesseract-ocr binary for pytesseract (e.g., apt install tesseract-ocr)
```

Example:

```
from pdf2image import convert_from_path
import pytesseract

images = convert_from_path("scanned.pdf", dpi=200)
text = ""
for img in images:
    text += pytesseract.image_to_string(img)
print(text)
```

Experiments

1. Convert at different DPIs (100, 200, 300) and compare OCR quality.
2. Crop an image to a smaller bounding box before OCR to reduce noise.

7. Export pipeline: `.ref` -> `.md` -> `.pdf`

This is practical: convert your `.ref` knowledge files to a styled PDF you can share.

Minimal flow idea (use markdown -> HTML -> PDF with WeasyPrint or wkhtmltopdf):

- `.ref` files are plain text with light markup (headings starting with `#`)
- Convert `.ref` -> `.md` (identity), or parse minimal markup to HTML

- Use `weasyprint` to render HTML to PDF (preserves CSS styling)

Example (using markdown and weasyprint):

```
pip install markdown weasyprint
```

Python sketch:

```
import markdown
from weasyprint import HTML
from pathlib import Path

text = Path("docs/pdf_basics.ref").read_text(encoding="utf-8")
html = markdown.markdown(text)
HTML(string=html).write_pdf("exported.pdf")
```

Experiments

1. Add a CSS style for `h1`, `h2` and render to see typography differences.
2. Add a page header/footer by wrapping HTML in `<header>`/`<footer>` and CSS `@page` rules.

8. Exercises (learning by doing — one small thing at a time)

Work through these in order. Re-type every example (don't copy/paste) and *explain* every line aloud or in a short note.

1. Minimal writer (FPDF):
 - Create `one_page.py` that writes "Hello" and saves. Remove `add_page()` and explain what happens.
2. Text wrapping:
 - Compare `cell` vs `multi_cell` with long text; write observations.

3. Fonts:

- Load a TTF font (`pdf.add_font("DejaVu", "", "DejaVuSans.ttf", uni=True)`) and use it to write non-ASCII text.

4. ReportLab precise layout:

- Create a 2-column layout using x coordinates.

5. Extract text:

- Use `pypdf` and `pdfplumber` on the same PDF and compare outputs; write a summary of differences.

6. Merge and split:

- Script that merges all `.pdf` in a folder.

7. OCR:

- Convert a scanned PDF and run `pytesseract`; compare OCR accuracy at different DPIs.

8. Build exporter:

- Implement `export_ref_to_pdf.py` that converts all `.ref` in `/docs` to `exports` using markdown -> weasyprint, and include a CSS file.

9. How to read a library's code & docs (practical steps)

Follow this short ritual to learn any library deeply:

1. Run the smallest example from docs.
2. Change one thing — read the effect.
3. Open the library in site-packages (or GitHub). Search for the function you used (e.g., `cell`).

4. Read the implementation, then follow internal helpers it calls.
 5. If it's C-extension code, focus on the Python wrapper and the design choices.
 6. Keep a one-page cheatsheet with real examples and mental notes.
-

10. Troubleshooting cheat-sheet (common gotchas)

- No text output from `extract_text()` → PDF might be image-based or text is drawn with custom encodings. Try pdfplumber or OCR.
 - Font not found in fpdf → add the TTF via `add_font()` with `uni=True` for unicode.
 - Bad layout in ReportLab → remember origin is bottom-left; y increases upwards.
 - WeasyPrint issues on Linux → ensure all system dependencies for rendering are installed (Cairo, Pango).
 - PyPDF merges produce corrupted file → always use binary modes and `PdfWriter()` and `write()` properly; ensure closing writers.
-

11. Minimal reference / cheatsheet

FPDF (fpdf2):

- FPDF(), add_page(), set_font(), cell(w,h,txt,ln), multi_cell(w,h,txt), image(path,x,y,w,h), output(path) ReportLab:
- canvas.Canvas(path,pagesize), drawString(x,y,txt), drawImage(path,x,y,w,h),

`showPage()`, `save()` pypdf / PyPDF2:

- `PdfReader(path).pages[], extract_text(), PdfWriter(), add_page(), write(fileobj)`
pdfplumber:
- `pdfplumber.open(path), page.extract_text(), page.extract_tables(),`
`page.extract_words()`

12. Next steps & learning schedule (two-week plan)

Week 1: basics and generation

- Day 1: FPDF minimal writer + experiments (2 hours)
- Day 2: Fonts & images, TTF load (1.5 hours)
- Day 3: ReportLab basics: coordinates, shapes (2 hours)
- Day 4: Build simple exporter (ref -> pdf using fpdf) (2 hours)
- Day 5: Refactor exporter, write README for yourself (1.5 hours)

Week 2: extraction and production workflows

- Day 6: pypdf text extraction + merging (2 hours)
- Day 7: pdfplumber tables (2 hours)
- Day 8: OCR experiments (2 hours)
- Day 9: Markdown -> PDF via weasyprint + CSS (2 hours)
- Day 10: Polish exporter, add tests, create sample .ref set (2 hours)

13. Final encouragement

You asked for something "from you" — this `.ref` is designed to be *your* learning artifact: retype, tinker, break, fix, and annotate it. The point is not to hoard perfect command snippets but to build muscle memory and intuition about how libraries think. After you complete the exercises, write your own compact cheatsheet — that sheet will be the knowledge you actually remember.

Appendix: Useful commands & installs

```
pip install fpdf2 reportlab pypdf pdfplumber pdf2image pytesseract markdown
weasyprint
```

System packages (Linux):

```
sudo apt install poppler-utils
tesseract-ocr libpq-dev libcairo2
libpango-1.0-0
libpangocairo-1.0-0
```
